Data Processing Tools
Part 1

Dr. Supaporn Erjongmanee

Department of Computer Engineering
Kasetsart University
fengspe@ku.ac.th

1

Course Description

#Credits (Lecture hours – Lab hours – Self-study hours)

- **01219116 Computer Programming II** 3(3-0-6)
- **01219117 Computer Programming Laboratory II** 1(0-3-2)

- Concepts and application of object-oriented programming. Inheritance and polymorphism. Modeling applications with objects. Recursion. Functional programming and first-class functions. Introduction to design patterns. Tools and techniques for data processing. Graphical user interface. Code quality for application development.

3

Scientific Python Ecosystem

- **Core numeric libraries**
  - Numpy: Numerical computing
  - Scipy: High-level numerical functions, e.g., optimization, regression
  - Matplotlib: 2D visualization
  - Sympy: Symbolic computing
- **Domain-specific packages**
  - Pandas, Statmodels, Seaborn: Statistics
  - Scikit-learn: Machine learning
  - Mayavi: 3D visualization
  - Scikit-image: Image processing

Image Source: https://scipy-lectures.org/intro/intro.html

Scientific Python Ecosystem (cont.)

- Library hierarchies

Core

Domain-Specific

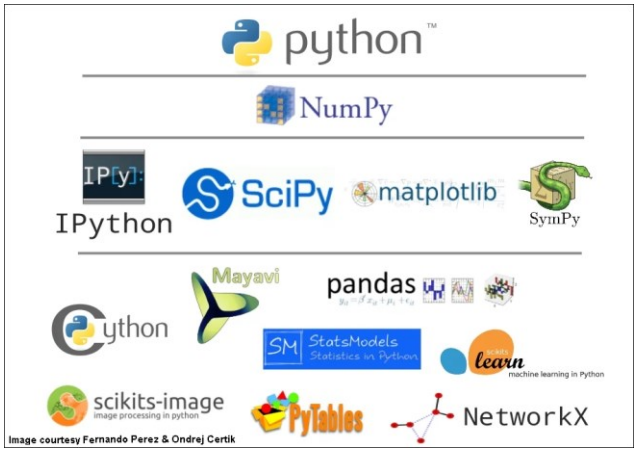

Image Source: https://speakerdeck.com/cournape/a-journey-through-the-scientific-python-ecosystem

Python Libraries for Data Science

- Top 10 Python libraries for Data Science

1.Tensorflow
2. NumPy
3. SciPy
4. Pandas
5. Matplotlib
6. Keras
7. SciKit-Learn
8. PyTorch
9. Scrappy
10. BeautifulSoup

1. NumPy
2. Keras
3. Pandas
4. PyTorch
5. SciPy
6. SciKit-Learn
7. Tensorflow
8. Matplotlib
9. Seaborn
10. Theano

1. Pandas
2. NumPy
3. SciPy
4. SciKit-Learn
5. Tensorflow
6. Keras
7. Matplotlib
8. Seaborn
9. Plotly
10. GGplot

1. NumPy
2. SciPy
3. BeautifulSoup
4. Scrappy
5. Tensorflow
6. Pandas
7. Matplotlib
8. Plotly
9. SciKit-Learn
10. PyCaret

Sources: https://www.simplilearn.com/top-python-libraries-for-data-science-article
https://builtin.com/data-science/python-libraries-data-science
https://www.geeksforgeeks.org/top-10-python-libraries-for-data-science-in-2021/
https://www.analyticsvidhya.com/blog/2020/11/top-13-python-libraries-every-data-science-aspirant-must-know-and-their-resources/
Image Source: https://makemeanalyst.com/data-science-with-python/python-libraries-for-data-analysis

Python Development Environments

- Pycharm
- Anaconda
- Spyder
- IPython
- Jupyter
- Google Colab

Numpy

- Shorten for “Numerical Python”
- Open-source Python library for scientific computation
- Examples
  - N-dimensional array handling
  - Complex functions such as basic math functions, linear algebras, random number generators

Image Source: https://numpy.org/

8

Numpy Import

- Numpy library must be imported in order to be used

```
import numpy
a = numpy.array([1,2,3])
a
```

- Numpy can be renamed using as keyword

```
import numpy as np
a = np.array([1,2,3])
a
```

Source: Numerical Processing and Basic Data Visualization, 01204111 Computers and Programming, https://www.cpe.ku.ac.th/~akrapong/204111/

9

Built-in Data Types

Numeric	int, float, complex
Text	str
Boolean	bool

- Numpy offers more data types than python
- Mostly, data types are varied by data size

Value	Numpy Data types
Integer	numpy.byte, numpy.short, numpy.intc, numpy.int_, numpy.longlong, numpy.ubyte, numpy.ushort, numpy.uintc, numpy.ulonglong numpy.int8, numpy.int16, numpy.int32, numpy.int64 numpy.uint8, numpy.uint16, numpy.uint32, numpy.uint64
Real number	numpy.half, numpy.single, numpy.double, numpy.longdouble numpy.csingle, numpy.cdouble, numpy.clongdouble numpy.float16, numpy.float32, numpy.float64, numpy.float96, numpy.float128 numpy.complex64, numpy.complex128, numpy.complex256 numpy.float_, numpy.complex_
Text	numpy.str_, numpy.unicode_ numpy.string_
Boolean	numpy.bool_, numpyt.bool8
Date & Time	numpy.datetime64, numpy.timedelta64

Source: https://numpy.org/doc/stable/reference/arrays.scalars.html#arrays-scalars-built-in

10

Built-in Data Types (cont.)

- Example

```
f = np.float_(1)
s = np.str_(1)
(f, s)
```

- We can use property dtype or dtype.name to find out data type of variable

```
f.dtype
```

```
s.dtype.name
```

Source: https://numpy.org/doc/stable/reference/arrays.scalars.html#arrays-scalars-built-in

11

Built-in Data Types (cont.)

- Example

```
f16 = np.float16(0.1)
f32 = np.float32(f16)
f64 = np.float64(f32)
(f16, f32, f64)
```

- We can use property astype to change variable data type

```
newf = f64.astype(np.float16)
newf.dtype
```

Source: https://numpy.org/doc/stable/reference/arrays.scalars.html#arrays-scalars-built-in

Array Creation

- Numpy offers Python-like list structure called array
  - Data type of this structure is called Numpy array
- numpy.array() function can be used to create multidimensional array
  - To create array with specific data type, such data type can be used as value of second parameter
- Example:

```
a1 = np.array([1,3,4,5])
a2 = np.array([1,3,4,5], dtype=np.str_)
a3 = np.array([1,3,4,5], dtype= np.complex_)
b = np.array([[2,3,4,5],[6,7,8,9]], dtype=float)
c = np.array([[[3.8,4,5,6],[7,8,9,10]],
              [[11,12,13,14],[15,16,17,18]],
              [[21,22,23,24],[25,26,27,28]]], dtype=np.int_)
```

Array Dimension

- To find out dimension of Numpy array, the following properties or function can be used
  - ndim
  - shape
  - size
  - len(array_name)
- Example:

```
c = np.array([[[3.8,4,5,6],[7,8,9,10]],
              [[11,12,13,14],[15,16,17,18]],
              [[21,22,23,24],[25,26,27,28]]], dtype=np.int_)
print(c.ndim)
print(c.shape)
print(len(c))
print(c.size)
```


Array Access

- Index of element inside Numpy array starts from zero.
- To access an element or subset of elements inside Numpy array, we use array_name[index]
- Example:

```
b = np.array([[2,3,4,5],[6,7,8,9]], dtype=float)
c = np.array([[[3.8,4,5,6],[7,8,9,10]],
              [[11,12,13,14],[15,16,17,18]],
              [[21,22,23,24],[25,26,27,28]]], dtype=np.int_)
```

```
print(b[1][2])
print(c[2][1][3])
```

```
print(b[1])
print(c[2])
print(c[2][0])
```

Array Slicing : Positive Index

- Slicing is to extract multiple elements of Numpy array at once using array_name[start_index : stop_index : step]
- The extracted array will not include element with stop_index
- If either start_index or stop_index or step is left out, the default value will be used.
  - Default value for start_index is zero.
  - Default value for stop_index is the last element index
  - Default value for step is one.
- Example:

Positive Index	0	1	2	3	4	5	6	7	8	9
Value	1	2	3	4	5	6	7	8	9	10

```
x = np.array([1,2,3,4,5,6,7,8,9,10], dtype=int)
print(x[2:8:2])
print(x[2:8])
print(x[2:])
print(x[:8])
```


Array Slicing : Negative Index

- Element index inside Numpy array can be negative, where first element has index = length -1 and last element has index = -1.
- If start_index and stop_index are left out, and step is -1, Numpy array will be reversed.
- Example:

Negative Index	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
Value	1	2	3	4	5	6	7	8	9	10

```
x = np.array([1,2,3,4,5,6,7,8,9,10], dtype=int)
print(x[-10:-5])
print(x[:-5])
print(x[-10:])
print(x[::-1])
```

Array Slicing : Multidimensional Array

- Example:

11	12	13	14	15	16	17	18
21	22	23	24	25	26	27	28
31	23	33	34	35	36	37	38

```
y = np.array([[11,12,13,14,15,16,17,18],
              [21,22,23,24,25,26,27,28],
              [31,32,33,34,35,36,37,38]], dtype=np.int_)
```

```
print(y[0, :])
```

```
print(y[0:2, 1:4])
```


Array Assignment, View and Copy

- Assignment (=) or view does not create new array
- To duplicate array, use copy()
- Example:

```
x = np.array([10,20,30])
y1 = x
y1[2] = 0
x
```

```
x = np.array([10,20,30])
y2 = x.view()
y2[2] = 0
x
```

```
x = np.array([10,20,30])
y3 = x.copy()
y3[2] = 0
x
```

Special Arrays

- Numpy offers special array where default data type is float.
  - numpy.ones: Array where all elements are ones.
  - numpy.zeros : Array where all elements are zeros.
  - numpy.eye: Identity matrix where diagonal elements are ones.
  - numpy.empty: Uninitialized array with specified size
- Example:

```
np.ones((2,3))
```

```
np.zeros((2,3), dtype=int)
```

```
np.eye(3, dtype=int)
```

```
np.empty([4, 2])
```

```
np.empty([5,2])
```


Array with Equally-Spaced Values

- We can use arange and linspace to generate array with equally-spaced values
  - numpy.arange(start, stop, step): Create a numpy array between start and stop (including start and excluding stop.) Difference between two values in the array is equal to step.
  - numpy.linspace(start, stop, n): Create a numpy array between start and stop (including start and stop.) There are n equally-spaced values inside the array.
  - Example:

```
x1 = np.arange(2,22,2)
```

```
x2 = np.arange(0,101,10)
```

```
y1 = np.linspace(0,1,5)
```

```
y2 = np.linspace(0,1,11)
```

Array with Randomized Values

- We can use numpy.random to generate array with randomized values
  - numpy.random.random(n): Generate n random numbers between [0, 1]
  - numpy.random.randint(maxInt, n): Generate n random integers between [0, maxInt]
  - numpy.random.{dist}({parameter_set}, n): Generate n random numbers based on distribution
    - See distribution list from https://numpy.org/doc/stable/reference/random/legacy.html#distributions
- Example:

```
x1 = np.random.random(10)
```

```
x2 = np.random.randint(100,size=10,dtype=int)
```

```
x3 = np.random.normal(5,2,size=10)  Normal distribution where mean = 5, SD = 2
```


Mathematic Operations

- For numpy, we can apply mathematic operations or functions to Numpy array
- Note that we cannot apply mathematic operations or functions to list directly.
  - We must apply to each element one at time.
- Example:

```
x = np.array([1,4,10])
y = np.array([2,3,6], dtype = float)
x*y
```

```
x = [1,4,10]
y = [2.0,3.0,6.0]
x*y
```

```
y-x
```

```
y-x
```

Mathematic Functions

- Numpy provides available math functions
  - Operations: numpy.add, numpy.substract, numpy.multiply, numpy.divide
  - Math functions: numpy.sqrt(), numpy.log(), numpy.sin()
  - Statistics functions: sum(), cumsum(), mean(), numpy.std(), min(), max(), sort()
- Example:

```
x = np.array([1,4,10])
y = np.array([2,3,6], dtype = float)
```

```
np.multiply(x,y)
```

```
np.subtract(y,x)
```

```
print(np.sqrt(x))
print(np.cos(x))
print(np.log(x))
print(np.exp(x))
```

```
print(np.sum(x))
print(np.cumsum(x))
print(np.diff(x))
print(np.min(x))
print(np.mean(x))
print(np.median(x))
print(np.var(x))
print(np.std(x))
```


Matrix Operations and Functions

- In numpy, Numpy 2D array can converted to numpy.matrix, and the matrix operation can be performed.
- Attribute of numpy.matrix such as I and T can help find inverse and transpose of matrix.
- Note that operation of Numpy 2D array is element-wise operation, not matrix operation
- Example:

```
x = np.matrix([[1,2],[3,4]])
y = np.matrix([[1,0],[0,1]])
```

```
x*y
```

```
x.T
```

```
x.I
```

```
a = np.array([[1,2],[3,4]])
b = np.array([[1,0],[0,1]])
```

```
a*b
```

```
a.T
```

```
a.I
```

Conclusion

- Data processing is to extract meaningful information from data
- Numpy is a python library to process data
- To handle Numpy, we must know
  - Data types
  - Data access
  - Special arrays
  - Mathematic and matrix operations and functions

Supaporn Erjongmanee
fengspe@ku.ac.th

Department of Computer Engineering
Kasetsart University